

Ilg'or RNN arxitekturalari: GRU va LSTM

Tabiiy tilni qayta ishlash — 8-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №8

26-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 GRU: axborot oqimini darvozalar boshqaradi
- 3 Vanishing gradient va cell state
- 4 LSTM darvozalari: forget, input, output
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa va amaliyotga ko'prik

Takrorlash: o'tgan darsdan ikki savol

1-savol

L7 da RNN gradienti uzoq qadamlarda **so'ndi**. Sababi nima edi? (Maslahat: tanh hosilasi va ko'paytma.)

2-savol

Bugun ertalab P7 da RNN klassifikatori qisqa sharhlarda ishladi. Uzoq, kesim oxiridagi gaplarda nega qiynaladi?

Takrorlash: o'tgan darsdan ikki savol

1-savol

L7 da RNN gradienti uzoq qadamlarda **so'ndi**. Sababi nima edi? (Maslahat: tanh hosilasi va ko'paytma.)

1-javob

$\partial h_t / \partial h_{t-1} = \text{diag}(1 - h^2) W_h$; har bo'g'in $\leq 1 \Rightarrow$ ko'paytma **eksponensial so'nadi**.

2-savol

Bugun ertalab P7 da RNN klassifikatori qisqa sharhlarda ishladi. Uzoq, kesim oxiridagi gaplarda nega qiynaladi?

2-javob

Uzoq bog'liqlik gradienti yo'qoladi $\Rightarrow$ model uzoqdagi so'zni **eslab qola olmaydi**. Bugun buni **darvozalar** bilan yengamiz.

Siz bugungi dars oxirida quyidagilarni bajara olasiz:

- ➊ **Tushuntira olasiz:** GRU update/reset darvozalari axborot oqimini qanday boshqaradi.
- ➋ **Tushuntira olasiz:** nega LSTM xotira katagi (cell state) vanishing gradientni yengadi (additiv “magistral”).
- ➌ **Hisoblay olasiz:** LSTM unutish darvozasini qo'lda — $f_t = \sigma(1.5) \approx 0.818$.
- ➍ **Taqqoslay olasiz:** GRU va LSTM ni (darvoza soni, parametr, qachon qaysi).

L7 dan: RNN rekurrensiyasi, vanishing gradient. Matematikadan: sigmoid σ , tanh, element-bo'yicha ko'paytma ($\odot$), zanjir qoidasi.

Bugungi dars rejasi:

Tsiki	Mavzu	Vaqt	Hosil natija	bo'luvchi
1	GRU: update/reset darvozalari	16 daq	$z = \sigma(1) = 0.731$	
2	Vanishing gradient va cell state	18 daq	0.9^{10} vs 0.42^{10}	
3	LSTM darvozalari (forget/input/output)	20 daq	$f_t = \sigma(1.5) = 0.818$	
4	GRU va LSTM ni taqqoslash	16 daq	parametr soni	
	Xulosa va 8-amaliyotga ko'rsatmalar	10 daq	Ertaga nima qurish-giz	

Bu dars RNN ni “tuzatadi”

L7 ning vanishing gradient muammosini darvozali xotira bilan yengamiz. Bugungi $f_t = \sigma(1.5) = 0.818$ ertangi P8 da assert bo'ladi.

Bugungi dars uchun muammo: o'tmishni o'zgarmagan holda olib yurish

Oddiy RNN qayerda qiynaladi:

Har qadam holatni qayta yozadi

$h_t = \tanh(W_h h_{t-1} + W_x x_t)$ — har qadam holatni **butunlay** yangilaydi va $\tanh$ siqadi. Eski ma'lumot tez “yuviladi”.

Hidden o'lchamini oshirish yordam bermaydi

Muammo o'lchamda emas — **ko'paytmali** gradient oqimida.

Bugungi yechim:

- 1 **Darvozalar** (gates): nimani saqlash, nimani yangilashni *o'rganadi*.
- 2 **Cell state** (LSTM): ma'lumotni *additiv* olib yuruvchi “magistral”.
- 3 Gradient $\tanh$ orqali emas, darvoza orqali oqadi $\Rightarrow$ so'nmaydi.

Ikki arxitektura: **GRU** (soddaroq) va **LSTM** (kuchliroq).

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 GRU: axborot oqimini darvozalar boshqaradi
- 3 Vanishing gradient va cell state
- 4 LSTM darvozalari: forget, input, output
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: eski va yangi holat orasida “aralashtirgich”

GRU har qadamda **qancha** yangilanishni darvoza bilan boshqaradi:

- **update gate** z : eskini saqlaymizmi yoki yangilaymizmi?
- **reset gate** r : eski holatning qanchasi yangi nomzodga kiradi?

$z \approx 0 \Rightarrow$ eski holat deyarli o'zgarmaydi (xotira saqlanadi).

Oddiy RNN har doim to'liq yangilaydi; GRU **tanlab** yangilaydi — shuning uchun uzoq xotira saqlanadi.

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}))$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

bunda: z_t — update gate (yangilash ulushi); r_t — reset gate; $\tilde{h}_t$ — nomzod holat; $\odot$ — element-bo'yicha ko'paytma; σ — sigmoid (0, 1). h_t — eski h_{t-1} va yangi $\tilde{h}_t$ ning **darvoza bilan** **vaznlangan** aralashmasi.

Keltirib chiqarish: GRU yangilanishi qayerdan keladi?

1-qadam. Oddiy RNN: $h_t = \tanh(\dots)$ — har doim **to'liq** qayta yoziladi.

Keltirib chiqarish: GRU yangilanishi qayerdan keladi?

1-qadam. Oddiy RNN: $h_t = \tanh(\dots)$ — har doim **to'liq** qayta yoziladi. **2-qadam.** G'oya: eski va yangi orasida **interpolyatsiya** qilaylik:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Keltirib chiqarish: GRU yangilanishi qayerdan keladi?

1-qadam. Oddiy RNN: $h_t = \tanh(\dots)$ — har doim **to'liq** qayta yoziladi. **2-qadam.** G'oya: eski va yangi orasida **interpolyatsiya** qilaylik:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

3-qadam. z_t ni **o'rganiladigan** darvoza qilamiz ($\sigma \in (0, 1)$). $z_t \rightarrow 0$: eski saqlanadi; $z_t \rightarrow 1$: yangiga o'tadi.

Xulosa: model **qachon** yangilashni o'zi o'rganadi — uzoq xotira uchun z_t kichik bo'lib qoladi.

Qo'lda misol: GRU update darvozasi

$$z_t = \sigma(W_z x_t + U_z h_{t-1}) = \sigma(1 \cdot 1.0 + 0) = \sigma(1.0).$$

Hisoblaymiz: ($h_{t-1} = 0.2$, $\tilde{h}_t = 0.8$)

$$z_t = \sigma(1.0) = 0.731$$

$$\begin{aligned} h_t &= (1 - 0.731) \cdot 0.2 + 0.731 \cdot 0.8 \\ &= 0.054 + 0.585 = \mathbf{0.639} \end{aligned}$$

$z = 0.731 \Rightarrow$ asosan yangi nomzodga o'tdi. z kichik bo'lsa edi, eski holat ko'proq saqlanardi.

Sizning vazifangiz: kichik update darvozasi

Savol — 60 soniya

Agar $z_t = \sigma(0) = 0.5$ bo'lsa ($h_{t-1} = 0.2$, $\tilde{h}_t = 0.8$):
 $h_t = ?$

Sizning vazifangiz: kichik update darvozasi

Savol — 60 soniya

Agar $z_t = \sigma(0) = 0.5$ bo'lsa ($h_{t-1} = 0.2$, $\tilde{h}_t = 0.8$):

$h_t = ?$

Javob

$$h_t = (1 - 0.5) \cdot 0.2 + 0.5 \cdot 0.8 = 0.1 + 0.4 = \mathbf{0.5}$$

$z = 0.5$ — eski va yangi teng aralashdi. $z \rightarrow 0$ bo'lsa $h_t \rightarrow h_{t-1}$ (xotira to'liq saqlanadi).

Kod $\leftrightarrow$ formula: GRU katagi

```
1 import numpy as np
2
3 def sigmoid(x):
4     return 1 / (1 + np.exp(-x))
5
6 def gru_step(h_prev, x, Wz, Uz, Wh, Uh, Wr, Ur)
7     :
8     z = sigmoid(Wz*x + Uz*h_prev)
9     r = sigmoid(Wr*x + Ur*h_prev)
10    h_tilde = np.tanh(Wh*x + Uh*(r*h_prev))
11    return (1 - z)*h_prev + z*h_tilde
12
13 print(gru_step(0.2, 1.0, 1,0, 1,0, 1,0))
```

Kod $\rightarrow$ formula:

Kod	Formula
z	z_t (update)
r	r_t (reset)
$(1-z)*h+z*ht$	h_t

Kaggle'da nn.GRU buni optimal C kodda bajaradi. P8 da m08 GRU ni sinaydi.

GRU tuzog'i: darvoza — sehr emas

Cheklovlar

- GRU vanishing gradientni **yumshatadi**, butunlay yo'qotmaydi.
- Juda uzoq bog'liqlikda hali ham qiynalishi mumkin.
- Ko'proq parametr $\Rightarrow$ kichik ma'lumotda ortiqcha moslashish.

Yechim yo'nalishi: alohida **cell state** bilan ma'lumotni additiv olib yurish — LSTM (2- va 3-tsikl).

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 GRU: axborot oqimini darvozalar boshqaradi
- 3 Vanishing gradient va cell state**
- 4 LSTM darvozalari: forget, input, output
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: ma'lumot uchun “magistral yo'l”

Oddiy RNN da har qadam ma'lumotni $\tanh$ va W_h orqali “qayta ishlaydi” — gradient siqiladi.

LSTM **cell state** c_t ni qo'shadi: ma'lumot unga **qo'shiladi** (additiv), darvozalar esa nimani o'chirish/qo'shishni boshqaradi.

c_t — to'siqsiz “magistral”; gradient bu yo'l bo'ylab so'nmasdan oqadi.

Tasvir: h_t — ko'cha (har burchakda to'xtash); c_t — avtomagistral (to'g'ridan-to'g'ri). Uzoq masofa uchun magistral kerak.

Cell state va gradient oqimi: rasmiy ta'rif

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad \frac{\partial c_t}{\partial c_{t-1}} = f_t$$

bunda: c_t — xotira katagi (cell state); f_t — forget gate (0, 1); i_t — input gate; $\tilde{c}_t$ — nomzod xotira; hosila f_t — na W matritsasi, na $\tanh$ siqilishi bor. Uzoq masofada gradient $\approx \prod_t f_t$: agar $f_t \approx 1$ bo'lsa, gradient **so'nmaydi** — oddiy RNN dagi $\prod(1 - h^2)W_h$ dan farqli.

Keltirib chiqarish: nega cell state gradientni saqlaydi?

1-qadam. Oddiy RNN: $\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h^2) W_h$ — har bo'g'in ≤ 1 , ko'paytma so'nadi.

Keltirib chiqarish: nega cell state gradientni saqlaydi?

1-qadam. Oddiy RNN: $\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h^2) W_h$ — har bo'g'in ≤ 1 , ko'paytma so'nadi.

2-qadam. LSTM cell: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Bu yerda c_{t-1} ga **matritsa ko'paytma yo'q**, faqat f_t ga ko'paytiriladi:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

Keltirib chiqarish: nega cell state gradientni saqlaydi?

1-qadam. Oddiy RNN: $\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h^2) W_h$ — har bo'g'in ≤ 1 , ko'paytma so'nadi.

2-qadam. LSTM cell: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Bu yerda c_{t-1} ga **matritsa ko'paytma yo'q**, faqat f_t ga ko'paytiriladi:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$

3-qadam. Uzoq masofa gradienti $\approx \prod_t f_t$. $f_t \approx 1$ bo'lsa, ko'paytma ≈ 1 — gradient saqlanadi.

Xulosa: additiv cell state + forget gate ≈ 1 — LSTM ning vanishing gradientga qarshi asosiy hiylasi.

Qo'lda misol: 10 qadamda gradient — RNN vs LSTM

RNN bo'g'ini $\approx 1 - h^2 = 0.42$ (L7); LSTM forget gate $f_t = 0.9$.

Oddiy RNN:

$$0.42^{10} \approx 1.7 \times 10^{-4}$$

Gradient deyarli **nol** — uzoq bog'liqlik yo'qoladi.

LSTM cell ($f = 0.9$):

$$0.9^{10} \approx \mathbf{0.349}$$

Gradient **saqlanadi** — 10 qadam uzoqlik ham ko'rinadi.

1.7×10^{-4} va 0.349 — ikki tartibdan ko'p farq. Mana shu farq LSTM ni uzoq matnlarda ishlatadi.

Sizning vazifangiz: 5 qadamli gradient

Savol — 60 soniya

LSTM forget gate $f = 0.9$, ketma-ketlik **5** qadam:

$\prod f = 0.9^5 = ?$ **(taxminan)**

Sizning vazifangiz: 5 qadamli gradient

Savol — 60 soniya

LSTM forget gate $f = 0.9$, ketma-ketlik **5** qadam:

$$\prod f = 0.9^5 = ? \quad (\text{taxminan})$$

Javob

$$0.9^5 = 0.59049 \approx \mathbf{0.59}$$

Oddiy RNN da $0.42^5 \approx 0.013$ bo'lardi. LSTM qisqa masofada ham gradientni ancha yaxshi saqlaydi.

Kod $\leftrightarrow$ formula: cell state oqimi

```
1 import numpy as np
2
3 # gradient oqimi: RNN vs LSTM cell (10 qadam)
4 rnn_factor = 0.42      # ~ (1 - h^2)
5 lstm_forget = 0.9      # forget gate
6
7 rnn_grad = rnn_factor ** 10
8 lstm_grad = lstm_forget ** 10
9
10 print("RNN :", rnn_grad)      # 1.7e-04
11 print("LSTM:", lstm_grad)     # 0.349
```

Kod $\rightarrow$ formula:

Kod	Formula
0.42**10	$\prod (1 - h^2)$
0.9**10	$\prod f_t$

Bu soddalashtirilgan model: amalda f_t har qadam o'zgaradi, ammo g'oya bir xil — additiv yo'l so'nishni kamaytiradi.

Cell state tuzog'i: forget gate 0 ga tushsa, xotira o'chadi

Forget gate ikki yoqlama

$f_t \approx 1$: xotira saqlanadi (yaxshi). $f_t \approx 0$: cell state **tozalanadi** — ba'zan kerak (yangi jumla), ba'zan ziyon. Modelni to'g'ri o'qitish muhim.

Amaliy maslahat: forget gate bias'ini musbat (masalan $+1$) boshlash — boshida xotirani saqlashga moyil qiladi (o'qitishni tezlashtiradi).

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 GRU: axborot oqimini darvozalar boshqaradi
- 3 Vanishing gradient va cell state
- 4 **LSTM darvozalari: forget, input, output**
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: uchta darvoza — o'chir, yoz, o'qi

LSTM cell state'ni **uch** darvoza boshqaradi:

- **forget** f_t : eski xotiradan nimani *o'chirish*.
- **input** i_t : yangi ma'lumotdan nimani *yozish*.
- **output** o_t : xotiradan nimani *o'qib* chiqarish (h_t ga).

Har biri $\sigma \in (0, 1)$ — “yarim ochiq eshik”.

Darvozalar — **o'rganiladigan** filtrlar.
Model qaysi ma'lumot muhimligini
ma'lumotdan o'rganadi.

LSTM: rasmiy ta'rif

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

$$h_t = o_t \odot \tanh(c_t)$$

bunda: $[h_{t-1}, x_t]$ — birlashtirilgan vektor; f_t, i_t, o_t — forget, input, output darvozalari; c_t — cell state; h_t — yashirin holat (chiqish).

Keltirib chiqarish: forget gate qanday ishlaydi?

1-qadam. Darvoza — $(0, 1)$ oralig'idagi “ulush”. Sigmoid aynan shuni beradi: $\sigma(\cdot) \in (0, 1)$.

Keltirib chiqarish: forget gate qanday ishlaydi?

1-qadam. Darvoza — $(0, 1)$ oralig'idagi “ulush”. Sigmoid aynan shuni beradi: $\sigma(\cdot) \in (0, 1)$.

2-qadam. Forget gate kirish va oldingi holatdan hisoblanadi:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Keltirib chiqarish: forget gate qanday ishlaydi?

1-qadam. Darvoza — $(0, 1)$ oralig'idagi “ulush”. Sigmoid aynan shuni beradi: $\sigma(\cdot) \in (0, 1)$.

2-qadam. Forget gate kirish va oldingi holatdan hisoblanadi:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

3-qadam. U cell state'ga element-bo'yicha ko'paytiriladi: $f_t \odot c_{t-1}$. f_t komponenti 1 ga yaqin — shu xotira saqlanadi; 0 ga yaqin — o'chiriladi.

Xulosa: forget gate — har bir xotira o'lchami uchun alohida “saqlash darajasi”. Endi uni qo'lda hisoblaymiz.

Qo'lda hisob: LSTM forget gate — $f_t = \sigma(1.5) = 0.818$

$$f_t = \sigma(W_f h_{t-1} + W_f x_t + b_f) \text{ (bir o'lchamli, soddalashtirilgan).}$$

Hisoblaymiz:

$$\begin{aligned} f_t &= \sigma(1 \cdot 0.5 + 1 \cdot 1.0 + 0) \\ &= \sigma(1.5) = \frac{1}{1 + e^{-1.5}} \\ &= \frac{1}{1 + 0.223} \approx \mathbf{0.818} \end{aligned}$$

$f_t = 0.818$ (1 ga yaqin) $\Rightarrow$ o'tmish holatining **ko'p qismi saqlanadi**.

Ertangi P8 birinchi assert:
`abs(f_t - 0.818) < 1e-3`
Ma'ruza L8 [13]-slayd

Sizning vazifangiz: cell state'ni yangilang

Savol — 90 soniya

$f_t = 0.818$, $c_{t-1} = 1.0$, input gate $i_t = 0.5$, nomzod $\tilde{c}_t = 0.6$:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t = ?$$

Sizning vazifangiz: cell state'ni yangilang

Savol — 90 soniya

$f_t = 0.818$, $c_{t-1} = 1.0$, input gate $i_t = 0.5$, nomzod $\tilde{c}_t = 0.6$:

$$c_t = f_t c_{t-1} + i_t \tilde{c}_t = ?$$

Javob

$$c_t = 0.818 \cdot 1.0 + 0.5 \cdot 0.6 = 0.818 + 0.30 = \mathbf{1.118}$$

Eski xotiradan 0.818 saqlandi, yangidan 0.30 qo'shildi — additiv yangilanish (cell state “magistral”i).

Kod $\leftrightarrow$ formula: LSTM katagi

```
import numpy as np
def sigmoid(x): return 1/(1+np.exp(-x))

def lstm_step(h, c, x, Wf, Wi, Wo, Wc):
    z = np.array([h, x])
    f = sigmoid(Wf @ z)           # forget
    i = sigmoid(Wi @ z)           # input
    o = sigmoid(Wo @ z)           # output
    c_tilde = np.tanh(Wc @ z)
    c = f*c + i*c_tilde           # cell state
    h = o*np.tanh(c)
    return h, c

# Wf=[1,1] -> f = sigmoid(0.5+1.0) = 0.818
```

Kod $\rightarrow$ formula:

Kod	Formula
f	f_t (forget)
i	i_t (input)
f*c+i*ct	c_t
o*tanh(c)	h_t

Kaggle'da nn.LSTM; P8 da m08 uni
GRU bilan taqqoslaydi.

LSTM tuzog'i: kuch — narx bilan keladi

Hisoblash narxi

- 4 ta darvoza/nomzod $\Rightarrow$ oddiy RNN dan $\sim 4\times$ ko'p parametr.
- Kichik ma'lumotda ortiqcha moslashish xavfi.
- Sekinroq o'qiydi (ko'proq hisob).

Yechim: kam ma'lumotda GRU (kamroq parametr) ko'pincha yetarli — keyingi tsiklda taqqoslaymiz.

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 GRU: axborot oqimini darvozalar boshqaradi
- 3 Vanishing gradient va cell state
- 4 LSTM darvozalari: forget, input, output
- 5 GRU va LSTM ni taqqoslash**
- 6 Xulosa va amaliyotga ko'prik

Intuitsiya: bir xil g'oya, ikki “vazn toifasi”

Ikkalasi ham darvozalar bilan uzoq xotirani saqlaydi:

- **LSTM**: alohida cell state, 3 darvoza — kuchliroq, og'irroq.
- **GRU**: cell state yo'q (h birlashgan), 2 darvoza — yengilroq, tezroq.

Ko'p vazifada natijalari yaqin — tanlov ma'lumot va resursga bog'liq.

Amaliy qoida: kam ma'lumot/tez kerak
⇒ GRU; ko'p ma'lumot/uzoq bog'liqlik
⇒ LSTM dan boshlang.

Parametr soni: rasmiy taqqoslash

$$\text{LSTM} \approx 4(d \cdot H + H \cdot H + H), \quad \text{GRU} \approx 3(d \cdot H + H \cdot H + H)$$

bunda: d — kirish o'lchami; H — yashirin o'lcham; 4 — LSTM ning 4 ta o'zgartirishi ($f, i, o, \tilde{c}$); 3 — GRU ning 3 tasi ($z, r, \tilde{h}$); $+H$ — bias. Demak $\text{GRU} \approx \frac{3}{4} \text{LSTM}$ parametri — shuning uchun tezroq va kam ma'lumotda barqarorroq.

Keltirib chiqarish: nega GRU kamroq parametr?

1-qadam. Har darvoza/nomzod bitta og'irlik to'plamiga ega: $d \cdot H$ (kirish) $+ H \cdot H$ (holat) $+ H$ (bias).

Keltirib chiqarish: nega GRU kamroq parametr?

1-qadam. Har darvoza/nomzod bitta og'irlik to'plamiga ega: $d \cdot H$ (kirish) $+ H \cdot H$ (holat) $+ H$ (bias). **2-qadam.** LSTM da bunday **4** ta blok bor ($f, i, o, \tilde{c}$); GRU da **3** ta ($z, r, \tilde{h}$).

Keltirib chiqarish: nega GRU kamroq parametr?

1-qadam. Har darvoza/nomzod bitta og'irlik to'plamiga ega: $d \cdot H$ (kirish) + $H \cdot H$ (holat) + H (bias). **2-qadam.** LSTM da bunday **4** ta blok bor ($f, i, o, \tilde{c}$); GRU da **3** ta ($z, r, \tilde{h}$).

3-qadam. Demak nisbat:

$$\frac{\text{GRU}}{\text{LSTM}} = \frac{3}{4} = 0.75$$

Xulosa: GRU LSTM parametrlarining $\sim 75\%$ i — arzonroq, lekin cell state'siz (biroz kamroq ifoda kuchi).

Qo'lda misol: parametr sonini sanaymiz

Bitta blok: $d \cdot H + H \cdot H + H = 2 \cdot 2 + 2 \cdot 2 + 2 = 10$.

Hisoblaymiz:

$$\text{LSTM} = 4 \times 10 = \mathbf{40}$$

$$\text{GRU} = 3 \times 10 = \mathbf{30}$$

GRU 30 vs LSTM 40 parametr — 25% kam.
Katta H da bu farq absolyut sonda kattalashadi.

Sizning vazifangiz: kattaroq model uchun parametr

Savol — 90 soniya

$d = 3$, $H = 2$ uchun bitta blok $= 3 \cdot 2 + 2 \cdot 2 + 2 = 12$.

LSTM va GRU parametrlari soni qancha?

Sizning vazifangiz: kattaroq model uchun parametr

Savol — 90 soniya

$d = 3$, $H = 2$ uchun bitta blok $= 3 \cdot 2 + 2 \cdot 2 + 2 = 12$.

LSTM va GRU parametrlari soni qancha?

Javob

$\text{LSTM} = 4 \times 12 = \mathbf{48}$; $\text{GRU} = 3 \times 12 = \mathbf{36}$.

Nisbat hamon $36/48 = 0.75$ — darvoza bloklari soni farqi (3 vs 4) saqlanadi.

Kod ↔ formula: PyTorch nn.LSTM / nn.GRU

```
1 import torch.nn as nn
2
3 # bir xil interfeys, faqat qatlam nomi farq qiladi
4 lstm = nn.LSTM(input_size=100, hidden_size=128,
5               batch_first=True)
6 gru = nn.GRU(input_size=100, hidden_size=128,
7             batch_first=True)
8
9 def n_params(m):
10     return sum(p.numel() for p in m.parameters())
11
12 print("LSTM:", n_params(lstm))      # ~117k
13 print("GRU :", n_params(gru))      # ~88k (~0.75x)
```

Kod → formula:

Kod	Formula
nn.LSTM	4 blok
nn.GRU	3 blok
n_params	$\approx 0.75 \times$

P8 da m08 aynan shu ikki qatlamni bir xil vazifada taqqoslaydi (compare_report).

Taqqoslash tuzog'i: "qaysi yaxshi" — noto'g'ri savol

Kontekstga bog'liq

Universal g'olib yo'q. Natija ma'lumot hajmi, ketma-ketlik uzunligi va hisoblash byudjetiga bog'liq. Bittasini "har doim yaxshi" deyish xato.

To'g'ri yondashuv: ikkalasini bir xil vazifada sinab, F1 va tezlikni **o'lchang** — aynan P8 da `compare_report` qiladi.

Prezentatsiya rejasi

- 1 Kirish: uzoq xotirani darvozalar bilan saqlaymiz
- 2 GRU: axborot oqimini darvozalar boshqaradi
- 3 Vanishing gradient va cell state
- 4 LSTM darvozalari: forget, input, output
- 5 GRU va LSTM ni taqqoslash
- 6 Xulosa va amaliyotga ko'prik

O'zbek tilida: cell state uzun qo'shimcha zanjirini ushlay oladimi?

1. Uzun qo'shimcha zanjiri:

o'rgan-il-gan-lar-dan — bir so'z, ≈ 4 morfema. Token yoki morfema darajasida bu uzun bog'liqlik.

LSTM afzalligi

Cell state morfema zanjiri davomida ma'noni **saqlay** oladi — oddiy RNN dan ustun.

2. SOV — kesim oxirida:

“men kitobni do'stinga berdim” — kesim uzoqda. LSTM cell state gap boshini oxirigacha eslab qola oladi.

Lekin

O'zbek korpuslari kichik $\Rightarrow$ LSTM ortiqcha moslashishi mumkin; GRU (kamroq parametr) ko'pincha barqarorroq.

Sintez: RNN, GRU, LSTM ni taqqoslaymiz

Model	Darvoza	Parametr	Vanishing'ga qarshilik
Oddiy RNN	0	eng kam	zaif ($\prod(1 - h^2)$)
GRU	2 (z, r)	$\sim 0.75 \times$ LSTM	yaxshi
LSTM	3 (f, i, o)	eng ko'p	eng yaxshi (cell state)

Asosiy g'oya

Darvozalar axborot oqimini **o'rganiladigan** tarzda boshqaradi; LSTM cell state additiv “magistral” bilan gradientni saqlaydi. GRU — shu g'oyaning yengil varianti. Tanlov vazifaga bog'liq.

Seminal manba: Hochreiter va Schmidhuber (1997)

Hochreiter, S., Schmidhuber, J. (1997). **Long Short-Term Memory**. *Neural Computation*, 9(8), 1735–1780.

Ahamiyati:

- Vanishing gradientga birinchi amaliy yechim — cell state va darvozalar.
- 20+ yil NLP/nutq tan olishda standart (Transformergacha).
- GRU (Cho va boshq., 2014) — shu g'oyaning soddalashtirilgani.

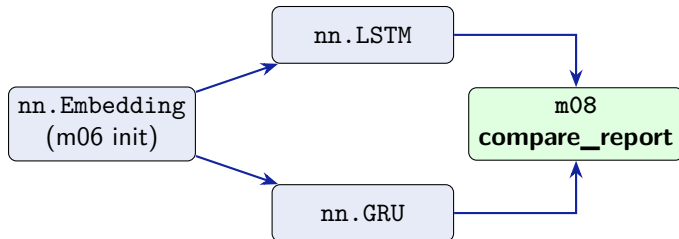
Muhokama savoli (P8 gacha o'ylang)

Kichik o'zbek korpusida GRU (kamroq parametr) va LSTM (kuchliroq xotira) — qaysi biri yaxshiroq ishlashi mumkin? Nega? (P8 da `compare_report` bilan tekshiramiz.)

Bugungi maqsadlar bajarildi

- ① ✓ **Tushuntirdik:** GRU update/reset darvozalari — $z = \sigma(1) = 0.731$ bilan interpolatsiya.
- ② ✓ **Tushuntirdik:** LSTM cell state additiv “magistral” — $0.9^{10} = 0.349$ vs RNN 1.7×10^{-4} .
- ③ ✓ **Hisobladik:** forget gate $f_t = \sigma(1.5) = 0.818$.
- ④ ✓ **Taqqosladik:** GRU $\sim 0.75 \times$ LSTM parametri (30 vs 40).

Ertaga P8: GRU va LSTM ni bir xil vazifada taqqoslaymiz



P8 da qurasiz:

- `m08 GRULSTMClassifier: arch='lstm' / 'gru'`
- Bir xil vazifada F1 va tezlik taqqoslash
- LSTM forget gate f_t ni tekshirish

Birinchi assert (bu slayd [I3]):

```
assert abs(f_t - 0.818)
      < 1e-3
```

*# Ma'ruza L8 [I3]-slayd bilan
solishtiring (forget gate)*

- ❶ Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation 9(8):1735–1780.
- ❷ Cho, K., va boshq. (2014). *Learning Phrase Representations using RNN Encoder-Decoder...* EMNLP 2014. (GRU.)
- ❸ Chung, J., va boshq. (2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. (GRU vs LSTM.)
- ❹ Jurafsky, D., Martin, J.H. (2023). *Speech and Language Processing*, 3-nashr. 9-bob: LSTM va GRU.
- ❺ `torch.nn.LSTM` / `torch.nn.GRU` hujjatlari.

Ilova S1: GRU — LSTM ning soddalashtirilgani

GRU LSTM dan ikki soddalashtirishda farq qiladi:

- Cell state c_t va yashirin holat h_t **birlashtirilgan** (bitta h_t).
- Forget va input darvozalari **bitta** update gate z_t ga birlashtirilgan: $h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$.

bunda: shu sababli GRU da 3 ta blok ($z, r, \tilde{h}$), LSTM da 4 ta ($f, i, o, \tilde{c}$).

Natija: kamroq parametr, tezroq o'qish; ko'p vazifada sifat farqi kichik.

Ilova S2: forget gate bias'ini musbat boshlash

Amaliy hiyla: b_f ni $+1$ (yoki $+2$) bilan boshlash $\Rightarrow$ boshida $f_t = \sigma(\text{katta}) \approx 1$, ya'ni model dastlab xotirani **saqlashga** moyil.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad b_f = +1$$

bunda: musbat bias o'qitish boshida vanishing gradientni yanada kamaytiradi — Jozefowicz va boshq. (2015) tavsiyasi.

Bu kichik o'zgartirish uzoq ketma-ketliklarda o'qishni sezilarli tezlashtiradi.